Navigating MLOps: Insights into Maturity, Lifecycle, Tools, and Careers

Jasper Stone*, Raj Patel†, Farbod Ghiasi‡, Sudip Mittal§, Shahram Rahimi¶
Dept. of Computer Science and Engineering, Mississippi State University
{jws819*, rkp88†, fg289‡}@msstate.edu, mittal@cse.msstate.edu§ rahimi@cse.msstate.edu¶

***Abstract*—The adoption of Machine Learning Operations (MLOps) enables automation and reliable model deployments across industries. However, differing MLOps lifecycle frameworks and maturity models proposed by industry, academia, and organizations have led to confusion regarding standard adoption practices. This paper introduces a unified MLOps lifecycle framework, further incorporating Large Language Model Operations (LLMOps), to address this gap. Additionally, we outlines key roles, tools, and costs associated with MLOps adoption at various maturity levels. By providing a standardized framework, we aim to help organizations clearly define and allocate the resources needed to implement MLOps effectively.**

***Index Terms*—Automated Machine Learning, Continuous Integration, DevOps, Large Language Models, Monitoring**

I. Introduction

Consumers increasingly expect AI-powered features in their daily interactions, ranging from weather forecasting to search and chat functionalities. Machine learning engineers and software developers must deploy these applications in scalable production-ready environments to ensure their responsiveness and availability. A study by Business Insights (2025) projects a 43% growth in the MLOps market within five years [1]. However, the study by an algorithmia analysis conducted in 2020 found that 55% of businesses using machine learning actively had not yet produced a model, and 18% required more than 90 days or more [2]. These prolonged deployment timelines illustrate the additional complexities associated with machine learning compared to traditional software and highlight the growing market demand for effective deployment solutions [3].

The continuous maintenance of machine learning models after deployment is necessary to address performance issues caused by data drift, unlike typical software [4]. Reproducibility is likewise difficult since small deviations in data or algorithms can produce inconsistent results, complicating model validation. Debugging and updates are difficult without automated deployment due to the differences in configurations across environments [5]. Monitoring of model accuracy and latency along with periodic retraining with new data is crucial to staying current [6]. These complexities affect manufacturing, business intelligence, aerospace and transportation among other industries.

Machine Learning Operations (MLOps) has emerged as a framework for deploying incremental machine learning improvements in a consistent and reliable manner. Despite the rise in MLOps adoption, there is no consensus among industries on what a comprehensive MLOps lifecycle framework entails [7] or on incorporating recent advancements in Large Language Models (LLMs) in the existing MLOps lifecycle. Given the recent rise in the popularity of LLMs, teams will be faced with the decision to train their own model or use an existing LLM. What criteria should be used to prefer one path over the other and how does this choice affect the MLOps lifecycle? Are there additional skills or technology needed if the team chooses to use a LLM?

As organizations embark on their MLOps journey, an evaluation of their MLOps maturity should be performed. MLOps maturity is defined as the qualitative assessment of people, processes, and technology involved in deploying and monitoring ML applications. At it's lowest level, maturity often involves manual creation and deployment, resulting in numerous challenges and limited observability. As maturity progresses, the deployment processes become more reliable, repeatable, and scalable. Highlighting the necessary roles and skill requirements allows teams to identify gaps and strategically plan for skill development and labor costs. Furthermore, identifying essential MLOps tools enables teams to align their skills and resource allocation.

This paper seeks to address questions about the lifecycle framework and maturity, providing insight into the personnel, tools, and planning required for successful implementation of MLOps. Therefore, our main contributions in this paper include:

- **Consolidated MLOps Lifecycle framework:** We propose a novel MLOps lifecycle framework combining best practices from academia and industry.
- **Inclusion of LLMOps:** Our proposed MLOps lifecycle framework includes LLMOps and the guidance for evaluating whether to either train your own LLM model or use an existing one.
- **Roles:** We provide a comprehensive list of roles involved throughout the MLOps lifecycle, from ideation through deployment and monitoring.
- **Tools:** An overview of tools supporting machine learning system development, including associated costs and alignment with MLOps lifecycle framework phases.

II. MLOps Lifecycle and The Taxonomy

Machine Learning Operations incorporates Software DevOps principles like Continuous Integration/Continuous Deployment (CI/CD) into machine learning workflows to improve collaboration, accelerate deployment, and ensure model

2025 IEEE Conference on Artificial Intelligence (CAI) | 979-8-3315-2400-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/CAI64502.2025.00118

979-8-3315-2400-5/25/$31.00 ©2025 IEEE
DOI 10.1109/CAI64502.2025.00118

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 15,2026 at 20:12:43 UTC from IEEE Xplore. Restrictions apply.

reproducibility. Introduced in 2015 [8], MLOps still lacks a standardized lifecycle, with various maturity models offering different stages and perspectives. Table I shows some maturity levels proposed by tech giants and researchers. These models generally begin with manual ML processes and evolve to full automation, covering data collection, model development, versioning, delivery, monitoring, and retraining, often with AutoML [7].

Company/Paper	Maturity Level Proposed(s)
Google [9]	Level 0: Manual Process Level 1: ML Pipeline Automation Level 2: Automated Training
Microsoft [9]	Level 0: No MLOps Level 1: DevOps but No MLOps Level 2: Automated Training Level 3: Automated Model Deployment Level 4: Full MLOps Automated Operations
Amazon [10]	Level 1: Initial Phase Level 2: Repeatable Phase Level 3: Reliable Phase Level 4: Scalable Phase
IBM [11]	Level 0: No MLOps Level 1: ML Pipeline Automation Level 2: CI/CD Pipeline Integration Level 3: Advanced MLOps
Meenu Mary et al. [4]	Level A: Automated Data Collection Level B: Automated Model Deployment Level C: Semi-automated Model Monitoring Level D: Fully-automated Model Monitoring

TABLE I: Recommended MLOps maturity levels in industry and research. Higher levels imply a higher level of maturity.

As MLOps process matures, aligning them with business needs becomes paramount to ensure that machine learning initiatives deliver tangible value. Figure 1 illustrates a MLOps maturity, where the highest level of automation and monitoring is implemented. The following description provides a detailed overview of our MLOps lifecycle framework, with terms in italics corresponding to those shown in Figure 1.

Business Needs (BN) take first priority, as MLOps is driven by the need to align machine learning initiatives with business objectives to address strategic challenges [12]. Data scientists set clear Objectives and Key Results (OKRs) focused on decision-making processes and critical operations [13]. Robust ML models are crucial for positive business outcomes [14] [15]. As organizations scale from proof-of-concept to operational machine learning, MLOps pipelines enable continuous retraining, deployment, and scalability [16]. Once business needs are identified, the next essential step is to collect and/or generate the data required to train models that support these objectives. Following business needs, *Data Collection/Generation* is central to machine learning. Data is sourced from proprietary databases, online repositories, sensors, or user-generated content [15], using methods such as web scraping, APIs, or crowdsourcing platforms such as Amazon Mechanical Turk [12] [16]. When data is limited, augmentation or transfer learning from pre-trained models may be employed [17], [18]. Data management systems oversee versioning, lineage, governance, and quality control, with collaboration tools such as JIRA and Slack that facilitate communication [16]. After gathering the necessary data, proper preparation techniques are employed to ensure the quality and consistency of inputs used for model development.

In *Data Preparation*, data is manipulated to improve model performance. This involves cleaning to handle missing values, errors, or duplicates [19], and normalizing input values to ensure consistency for algorithm efficiency [15], [19]. Data augmentation techniques, such as rotation and scaling, are applied to artificially expand datasets [17]. Integration of multiple data sources and careful versioning ensure reproducibility and consistency throughout the model lifecycle [16]. Once the business needs have been identified and data collected, we move into administrative setup to create the environments for the team to collaborate.

Establishing a robust administrative framework in the *Administrative & Setup (AS)* phase is crucial for managing and streamlining MLOps workflows. This involves setting up project and versioning repositories, machine learning workspaces, defining user roles and access controls, and scaling hardware to accommodate varying demands. *Project Repositories* are used to facilitate collaborative work on code, models, and data. This ensures that team members can collaborate, track changes, experiment and backup their work. Following this, machine learning workspaces are configured. *Machine Learning Workspaces* are created for model development. These workspaces leverage platforms like MLflow [20], Kubeflow [21], JupyterHub, and Databricks [22] to centralize tools, datasets, and computational resources, streamlining workflows and boosting team productivity. After establishing workspaces, environments for data ingestion and transformation are set up using tools like Labelbox or Amazon SageMaker Ground Truth [23]. Accurate labeling is essential for effective model training. Following data labeling, datasets are stored and versioned. *Datasets* are structured and managed to ensure data integrity, confidentiality, and compliance with data protection regulations. Proper dataset management supports the project lifecycle and ensures data quality for model training.

Provisioning *Compute Resources* (CPU, GPU, TPU) is essential for model training and deployment. This includes both cloud-based and on-premises resources to meet computational demands. Adequate compute resources are crucial for efficient model experimentation and deployment. *Defining Users and Access* involves setting permissions to ensure secure and efficient collaboration. Tools such as Identity and Access Management (IAM) are employed to manage access controls, ensuring security and data integrity [24]. Administrators *Create Monitors* using tools like Prometheus and Elasticsearch, Logstash, Kibana (ELK) Stacks, are established to track system performance and detect anomalies. Effective monitoring is critical for maintaining system health and ensuring smooth operation throughout the MLOps lifecycle framework. Once the administrative setup is in place, it is crucial to implement robust version control mechanisms to track changes and ensure

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 15,2026 at 20:12:43 UTC from IEEE Xplore. Restrictions apply.

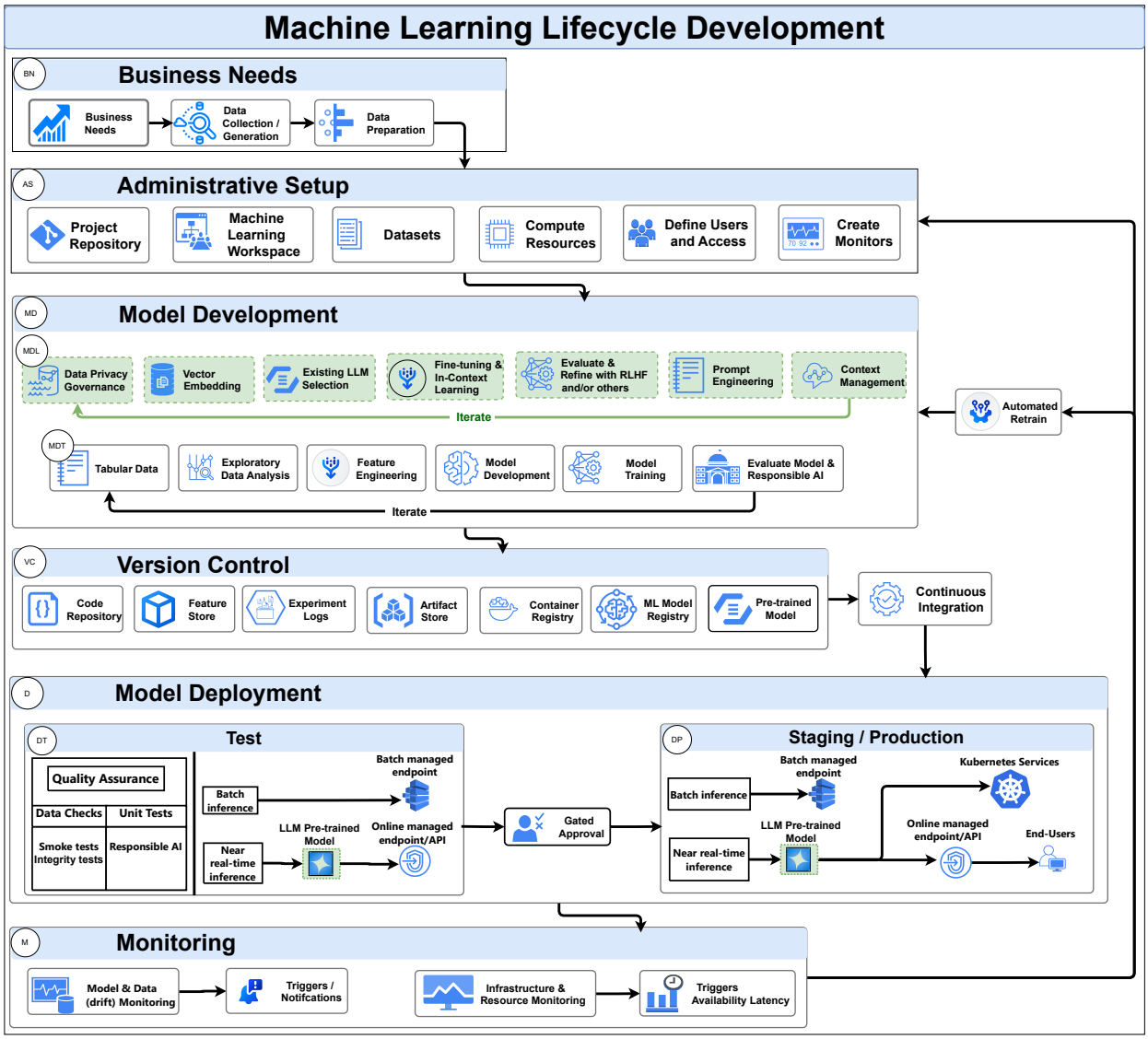

Fig. 1: The consolidated MLOps lifecycle begins at the top-left and progresses left to right within each phase. The forest-green highlighted pipeline represents the LLMOps development cycle and is not complete as it needs other phases as well. Each completed phase leads downward to monitoring, whose feedback loops back into administrative setup and model development, enabling iterative improvement.

reproducibility throughout the pipeline.

During *Model Development (MD)*, a decision must be made to either leverage an existing large language model (LLM) or train a custom model. Unless your application needs a specialized model and have access to ample high-quality data, significant computational resources, specialized machine learning expertise, and can continuously maintain the model, using an existing LLM is generally preferable, as illustrated in Figure 2. Utilizing a pre-existing model allows your team to build upon previous innovations, significantly accelerating deployment. The *(MDL)* stage within model development, highlighted in forest green with dotted lines in Figure 1, encompasses several key tasks: *Data Privacy Governance* focuses on eliminating any data that could violate privacy or governance regulations, such as GDPR and CCPA [25]. *Vector Embedding* involves storing high-dimensional data efficiently as vectors, particularly beneficial for LLMOps tasks [25]. *Existing LLM Selection* process requires identifying suitable models such as LLaMA, GPT-J, Flan-T5 or Stable Diffusion [25] to address specific business tasks. *Fine Tuning and In-Context Learning* involves adapting existing LLMs via techniques such as zero-shot prompting, where only the prompt

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 15,2026 at 20:12:43 UTC from IEEE Xplore. Restrictions apply.

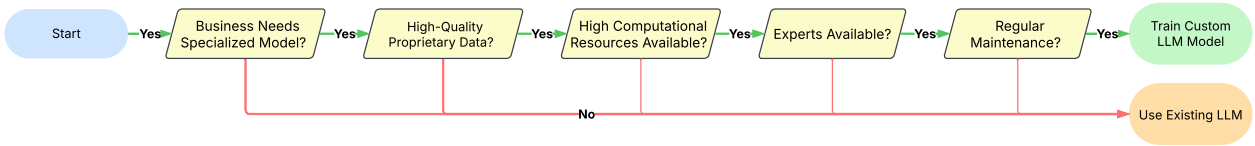

Fig. 2: Recommended decision process for training your own custom LLM model vs using an existing LLM.

is given, or few-shot prompting, where examples are provided for the model's context. Fine-tuning adjusts weights in the model's final layers, resulting in a smaller, optimized language model ready for deployment. This approach enhances accuracy, reduces latency, and improves cost efficiency. One such example is to train a model to generate sarcastic responses [26]. In-context learning (ICL) involves inserting context data directly into the LLM prior to the user's prompt, enabling the model to reason effectively. Practical examples include document summarization or embedding data chunks from retrieval-augmented generation (RAG) systems.

Evaluate & Refine with RLHF and/or others entails tuning parameters such as temperature, max-tokens, top-k, top-p, frequency penalty, and presence penalty, each affecting creativity, response length, and predictability. Model evaluation in LLMOps typically employs intrinsic metrics like ROUGE, BERT, and BLEU scores, assessing how closely outputs match reference answers. Human evaluation involves experts or crowdsourced evaluators examining LLM performance in specific scenarios. Task-specific benchmarks like GLUE or SuperGLUE offer standardized tasks and metrics for comprehensive assessment [27]. Additionally, unit and integration testing tools such as DeepEval [28], BLEU [29], and ROUGE [29] verify model performance. Reassessing Fairness, Accountability, Transparency, and Ethics (FATE) [30] ensures that fine-tuning and prompt engineering have not introduced bias or other undesirable outcomes. Occasionally, models are further refined through reinforcement learning with human feedback (RLHF) [29]. *Prompt Engineering* is the strategic crafting of prompts to guide an LLM toward desired outputs. Techniques include zero-shot, few-shot, chain-of-thought, and prompt-chaining [6] [31]. *Context Management* involves effectively managing relevant contextual information to ensure the accuracy and relevance of LLM responses prior to deployment.

The *(MDT)* section of model development, shown in solid lines, focuses on iteratively building and refining machine learning models. This includes working with different data types, algorithm selection, and feature engineering. The first step is to ensure the data is in a format that facilitates machine learning. *Tabular Data* structures are well-suited for various machine learning algorithms, revealing hidden patterns through de-normalization [16]. Despite potential data redundancy, this format enhances data retrieval and supports effective pre-processing, feature engineering, and visualization, crucial for improving model performance and facilitating *Exploratory Data Analysis (EDA)* [13]. EDA involves using visualization tools like histograms and scatter plots to explore correlations, patterns, and anomalies within the data [4]. By summarizing key characteristics through statistical methods, EDA helps detect outliers and guide feature selection thus improving model accuracy [3]. *Feature Engineering* transforms raw data into meaningful inputs for models, employing techniques like Principal Component Analysis (PCA) for dimensionality reduction and hyper-parameter tuning for model optimization [16] [17]. Creating new features from existing data through techniques like polynomial features enhances model robustness and accuracy [17].

Model Development involves selecting the appropriate algorithm, such as linear regression, decision trees, or neural networks, depending on the data and problem context [13] [17]. Ensuring model interpretability and explainability is crucial for building stakeholder trust, while evaluation metrics like precision and F1-score help confirm alignment with business goals [18] [13]. *Model Training* involves feeding features and algorithms into the model to learn patterns and relationships within the training data, adjusting parameters to minimize prediction errors [32]. Cross-validation ensures generalization and prevents overfitting [18] [3]. Complex models often require GPUs or TPUs, with early stopping and learning rate adjustments to optimize the process [16]. After training, the team will *Evaluate Model & Responsible AI* utilizes metrics such as ROC-AUC and mean squared error to assess performance on test data [17]. Responsible AI practices ensure fairness by mitigating biases through fairness metrics and continuous monitoring, ensuring ethical and robust model deployment [32] [18]. While model development is a core element, it must be supported by an efficient administrative framework and infrastructure setup to ensure smooth operations.

Version Control (VC) is crucial in machine learning for ensuring consistent performance and reproducibility by tracking parameters like data, features, hyperparameters, and source code. Versioning enables reliable model reproduction and maintains experiment integrity and auditability through tools like DVC [33] and Git [34] [3] [17]. Model versions, metadata, training configurations, and performance metrics are also tracked, enabling teams to revert to earlier versions if performance declines, thus preserving system reliability. With version control established, continuous integration (CI) systems help automate testing, ensuring new versions of the model are validated efficiently.

Continuous Integration (CI) automates testing and validation of new code and model versions, ensuring system stability. Automated tests, triggered when a new model version is added, assess performance, code quality, and integration issues. CI integrates with version control systems to automatically update code and data, ensuring that all changes are thoroughly tested

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 15,2026 at 20:12:43 UTC from IEEE Xplore. Restrictions apply.

Aspect	MLOps	LLMOps
Computational Demands	Uses standard CPUs or limited GPU resources.	Relies heavily on high-performance GPUs or TPUs for both training and inference.
Cost Structure	Driven largely by experimentation, data collection, and initial preprocessing.	Dominated by GPU-based inference, high computational costs, and licensing fees for proprietary models.
Data Management	Requires extensive preprocessing of large-scale, labeled datasets.	Involves carefully curated datasets, enabling fine-tuning and effective prompt engineering even with smaller volumes of data.
Experimentation & Hyperparameter Tuning	Focuses extensively on experimentation to optimize accuracy, performance, and hyperparameter adjustments.	Primarily emphasizes fine-tuning foundation models, refining prompts, and balancing performance, computational efficiency, and costs.
Transfer Learning	Minimal transfer learning; often involves training models from scratch.	Heavily depends on transfer learning through fine-tuning of pre-trained foundation models.
Model Evaluation & Human Feedback	Relies on standard metrics (accuracy, precision, F1-score), with minimal human involvement.	Utilizes specialized metrics (e.g., BLEU, ROUGE, DeepEval, FATE), frequently incorporating extensive human feedback, particularly Reinforcement Learning with Human Feedback (RLHF).
Prompt Engineering	Typically not applicable due to structured data and defined tasks.	Crucial for optimizing performance, reducing hallucination, and ensuring reliable, high-quality responses.
Deployment Complexity & Latency	Simple deployment on standard infrastructure, moderate latency concerns.	Requires dedicated GPU infrastructure, complex deployment pipelines (e.g., LangChain, LlamaIndex), and significant latency challenges impacting real-time responsiveness.
Governance & Monitoring	Governance and monitoring practices are comparatively straightforward, involving standard oversight and regulatory measures.	Heightened risks of hallucination, data leakage, and misuse necessitate rigorous, advanced governance frameworks and continuous model monitoring.

TABLE II: Comparison of MLOps and LLMOps derived from Section II.

before merging [35] [9].

Model Deployment (D) involves comprehensive testing, validation, and monitoring before and after production deployment. In the *Test* envinronment, the model is tested in production-like conditions to validate its performance. Quality assurance tests, including batch and near real-time inference, assess large data processing and time-sensitive accuracy. Responsible AI checks prevent bias, while integration tests detect critical issues early, ensuring the model's reliability before deployment [87]. *Gated Approval* in MLOps ensures approved model deployment by requiring authorization before models progress to staging or production. Requiring gated approval reduces the risk that an individual pushes software and resources into production before they are fully vetted and approved by the appropriate personnel. In *Staging/Production*, once the model passes quality assurance tests, the entire application is deployed for end-user access, supporting both batch and near real-time inferencing. Performance benchmarks, such as the MLPerf Inference Benchmark presented in Reddi's work (2020), can be used to evaluate both inference types, confirming production readiness, ensuring smooth transitions, and preserving accurate predictions. [88].

Continuous *Monitoring (M)* aggregates insights from test, staging, and production environments to track performance metrics. This system enables adaptive responses to operational issues, especially in multi-tenant environments. Any deviations trigger adjustments to ensure that the model continues to meet business requirements [89].

In summary, MLOps lifecycle framework establishes a comprehensive, scalable system for managing machine learning lifecycles, facilitating smooth transitions between development, deployment, and monitoring. The integration of tools such as version control, CI/CD pipelines, and responsible AI ensures operational efficiency and ethical model performance.

III. MLOps Roles and Tools

The previous section describes a breakdown of each phase and the associated tasks in the MLOps lifecycle framework. We now turn our focus to the roles and tools that are involved. The roles, emphasized in italllics, can be found in Table III. The comprehensive list of tools are presented in Table IV.

The first priority is to align business needs and Key Performance Indicators (KPIs) with the effort to build and deploy a model. It's the job of the *Business Analysts* [36] to focus on the strategic side of MLOps. They align the MLOps process with business objectives by translating business needs into epics and milestones using tools like Azure DevOps, Wekan [38] or Taiga [39]. The rest of the team will break down these large long term goals into smaller short term technical goals. Given the established business strategy, *System Administrators* and *Network Administrators* [36] will design, create and maintain the cloud and on prem infrastructure using tools like Ansible [45], Terraform [46] and Bicep [48]. Once the infrastructure is created and proper access has been verified by the team, *Data Architects* and *Data Engineers* [36] can get to work on ingesting, transforming, storing and versioning the data. Databricks [22] and Azure ML Studio [40] are examples of tools that both Data Engineers and *Data Scientists* use but Data Scientists will also analyze, visualize and build statistical models with Python and R. *ML Engineers* can then take the prepared data to create and/or fine tune models with PyTorch [51], Tensorflow [52], Azure ML Studio and MLFlow [20]. Rather than building a model from scratch, a *Prompt Engineer* could design a LLM prompt to achieve the desired output. Training and tuning iterations are stored in MLFlow and models are stored in repositories like Azure ML Studio and

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 15,2026 at 20:12:43 UTC from IEEE Xplore. Restrictions apply.

MLOps Roles	MLOps Abbreviated Phase	DOL Code	Department Of Labor (DOL) Title	Monthly Wage Estimate DOL 2023
Business Analyst	BN, M	13-1199	Business Operations Specialists	$7,427 [36]
System Administrator	AS, VC, D, M	15-1244	Network and Computer Systems Administrators	$8,381 [36]
Network Administrator	AS, VC, D, M	15-1244	Network and Computer Systems Administrators	$8,381 [36]
Data Architect	BN, MD, M	15-1243	Database Architects	$11,419 [36]
Data Engineer	BN, MD, M	15-2051	Data Scientists	$9,920 [36]
Data Scientist	BN, MD, M	15-2051	Data Scientists	$9,920 [36]
ML Engineer	MD, D, M	15-1299	Computer Occupations, All Other	$9,369 [36]
Prompt Engineer	MDL, D, M	15-1299	Computer Occupations, All Other	$9,369 [36]
Software Engineer	AS, MD, VC, D, M	15-1252	Software Developers	$11,509 [36]
QA Engineer	DT, VC	15-1253	Software Quality Assurance Analysts	$9,038 [36]
MLOps Engineer	AS, MD, VC, D, M	15-1252	Software Developers	$11,509 [36]
DevOps Engineer	AS, MD, VC, D, M	15-1252	Software Developers	$11,509 [36]
Security and Compliance Officer	BN, AS, MD, VC, D, M	15-1212	Information Security Analysts	$10,395 [36]

TABLE III: MLOps roles mapped to corresponding department of labor codes. The MLOps phase abbreviations can be found in Figure 1. Some roles were not an exact match for an existing department of labor occupation so the next closest alternative was selected.

Tool	Cost/Month	Open Source	BN	AS	MDT	MDL	VC	CI	D	M
Delta Lake [37], Wekan [38]	$0	✓	✓							
Taiga [39]	$70	✓	✓							
Databricks [22]	CB	✓	✓							
Azure ML Studio [40]	$0		✓		✓	✓	✓		✓	
AWS Lake Formation [41]	CB		✓	✓	✓	✓				
Asana [42]	$110		✓							
Clickup [43], Trello [44]	$0		✓							
Ansible [45], Terraform [46], ARM [47], Bicep [48]	$0	✓		✓						
Chef [49] and Puppet [50]	CB	✓		✓				✓		
PyTorch [51], Tensorflow [52], Pandas [53]	$0	✓			✓					
JUnit [54], PyTest [55]	$0	✓			✓	✓				
Kubeflow [21]	$0	✓			✓	✓			✓	✓
Composer by MosaicML [56], H2O.ai [57]	CB	✓			✓					
AI Fairness 360 [58]	$0	✓			✓	✓				✓
AWS Sagemaker [59]	CB				✓	✓			✓	✓
AWS Kinesis [60]	CB				✓					
Neptune.ai [61], Weights & Biases [62]	$500	✓			✓	✓				
LangChain [63], LlamaIndex [64], DeepAI [65]	$0	✓				✓				
LMStudio [66]	By Request	✓				✓				
Git [34], DVC [33], FEAST [67], Container Registry [68], NVIDIA NGC Catalog [69]	$0	✓					✓			
Bitbucket [70]	$33						✓			
Azure Container Registry [71]	$5						✓			
MLFlow [20]	$0	✓			✓	✓	✓	✓		
Hugging Face [72]	$0	✓			✓	✓	✓		✓	
Azure DevOps [73]	$30		✓				✓	✓	✓	
GitLab Enterprise [74]	$290	✓	✓				✓	✓	✓	
OpenLLM [75], TensorFlow Serving [76]	$0	✓							✓	
Jenkins [77]	$0	✓						✓		
Seldon [78]	$0	✓			✓		✓		✓	✓
ELK Stack [79], Prometheus [80], Grafana [81], OpenLLMetry [82]	$0	✓								✓
Grafana Cloud [83]	$19	✓								✓
Whylabs [84]	$250	✓								✓
Datadog [85]	CB	✓								✓
Langsmith [86]	$390	✓				✓	✓			✓

TABLE IV: Tools that are used in each MLOps lifecycle phase along with their monthly cost. The MLOps phase abbreviations can be found in Figure 1. Consumption based pricing (CB) is for services priced by consumption instead of subscription. The cost per months is based on 10 users.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 15,2026 at 20:12:43 UTC from IEEE Xplore. Restrictions apply.

Huggingface. *Software Engineers* then develop applications that interact with the models to deliver value to customers using their favorite Integrated Development Environment(IDE) and versioning their source code with Git or Bitbucket. A *QA Engineer* [36] will use testing frameworks like JUnit [54] or PyTest [55] to ensure the software performs as defined in the feature request. At this point everything is tied together by *DevOps Engineers* and *MLOps Engineers* [36] to ensure the software and models are built, tested and deployed onto the prepared infrastructure in a reliable and reproducible manner. The appropriate model version is paired with a compatible source code version and built into a deployable image that can be stored in a Container Registry and deployed into the cloud or into a Kubeflow cluster. The deployment pipelins should also ensure automated configuration of logging, traces and metrics for monitoring. Finally, *Security and Compliance Officers* play a key role in ensuring that the entire process adheres to legal standards and remains protected from unauthorized access.

The creation of an MLOps pipeline involves a variety of tools across different phases of the machine learning lifecycle framework, ranging from data collection to model development to deployment and monitoring. Each tool plays a critical role in ensuring that the entire process runs efficiently, and the flexibility in choosing tools is key to customizing the pipeline based on team skills, infrastructure, and specific project needs.

IV. Conclusion

This paper covers several existing MLOps maturity levels and proposes a consolidated lifecycle framework that includes LLMOps along with the roles and tools involved. A consolidated framework creates a shared understanding and common terminology, building on lessons learned from similar challenges. Mapping roles and tools to MLOps phases aids in cost estimation and resource allocation. As teams adopt MLOps practices, the deployments of machine learning applications become more reliable, reproducible, scalable, and observable. The lifecycle framework also serves as a foundation for iterative improvement as MLOps principles evolve. However, there are several areas that could benefit from additional research such as

1) A case study to apply the framework to a new AI-enabled application project, tracking implementation time, costs, and tools involved.
2) Review and highlight improvements to our framework, particularly with the integration of LLMOps.
3) A comprehensive survey of the threat landscape that identifies potential attacks and challenges in securing MLOps, while also recommending strategies for mitigating these threats.
4) Determine safeguards to protect AI-integrated edge devices, such as security cameras, autonomous vehicles, and healthcare devices, from data poisoning and cyber threats?

Although this paper addresses several improvements, investigating security implications while implementing the MLOps lifecycle framework is an area that would benefit from future research. As organizations strive to achieve more reliable and consistent model deployments, this work builds on the foundation of other MLOps research and paves the way for future advancements.

Acknowledgment

This work was supported by the Predictive Analytics and Technology Integration (PATENT) Laboratory at the Department of Computer Science and Engineering, Mississippi State University.

References

[1] Fortune Business Insights, "MLOps Market Size, Share & Forecast — Global Report [2030]." https://www.fortunebusinessinsights.com/mlops-market-108986, 2025. Accessed: 2025-03-07.
[2] "2020 state of enterprise machine learning." https://cdn2.hubspot.net/hubfs/2631050/0284%20CDAO%20FS/Algorithmia_2020_State_of_Enterprise_ML.pdf, 2020. Accessed: 2024-11-04.
[3] A. Lima, L. Monteiro, and A. P. Furtado, "Mlops: Practices, maturity models, roles, tools, and challenges - a systematic literature review," vol. 1, pp. 308–320, Science and Technology Publications, Lda, 2022.
[4] M. M. John, H. H. Olsson, and J. Bosch, "Towards mlops: A framework and maturity model," pp. 334–341, Institute of Electrical and Electronics Engineers Inc., 9 2021.
[5] N. Hewage and D. Meedeniya, "Machine learning operations: A survey on mlops tool support," 2 2022.
[6] R. Shan and T. Shan, "Enterprise llmops: Advancing large language models operations practice," in *Proceeding - 2024 IEEE Cloud Summit, Cloud Summit 2024*, pp. 143–148, Institute of Electrical and Electronics Engineers Inc., 2024.
[7] G. Symeonidis, E. Nerantzis, A. Kazakis, and G. A. Papakostas, "Mlops - definitions, tools and challenges," pp. 453–460, Institute of Electrical and Electronics Engineers Inc., 2022.
[8] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems."
[9] "Mlops: Continuous delivery and automation pipelines in machine learning — cloud architecture center — google cloud," 5 2023.
[10] S. Kartakis, G. A. Porcelli, G. Schinas, and S. Eigenbrode, "Mlops foundation roadmap for enterprises with amazon sagemaker — aws machine learning blog," 6 2022.
[11] T. Mucci and C. Stryker, "What is MLOps? — IBM," 4 2024.
[12] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Computer security–ESORICs 2020: 25th European symposium on research in computer security, ESORICs 2020, guildford, UK, September 14–18, 2020, proceedings, part i 25*, pp. 480–501, Springer, 2020.
[13] M. Testi, M. Ballabio, E. Frontoni, G. Iannello, S. Moccia, P. Soda, and G. Vessio, "Mlops: a taxonomy and a methodology," *IEEE Access*, vol. 10, pp. 63606–63618, 2022.
[14] N. Pitropakis, E. Panaousis, T. Giannetsos, E. Anastasiadis, and G. Loukas, "A taxonomy and survey of attacks against machine learning," *Computer Science Review*, vol. 34, p. 100199, 2019.
[15] F. A. Yerlikaya and Ş. Bahtiyar, "Data poisoning attacks against machine learning algorithms," *Expert Systems with Applications*, vol. 208, p. 118101, 2022.
[16] S. Mäkinen, H. Skogström, E. Laaksonen, and T. Mikkonen, "Who needs mlops: What data scientists seek to accomplish and how can mlops help?," in *2021 IEEE/ACM 1st Workshop on AI Engineering-Software Engineering for AI (WAIN)*, pp. 109–112, IEEE, 2021.
[17] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (mlops): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023.
[18] X. Zhang and J. Jaskolka, "Conceptualizing the secure machine learning operations (secmlops) paradigm," in *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*, pp. 127–138, IEEE, 2022.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 15,2026 at 20:12:43 UTC from IEEE Xplore. Restrictions apply.

[19] I. Kumara, R. Arts, D. Di Nucci, W. J. Van Den Heuvel, and D. A. Tamburri, "Requirements and reference architecture for mlops: Insights from industry," 2022.
[20] MLflow, "Mlflow." https://mlflow.org/, 2024. [Accessed 16-12-2024].
[21] Kubeflow, "Kubeflow." https://www.kubeflow.org/, 2024. [Accessed 16-12-2024].
[22] Databricks, "Databricks." https://www.databricks.com/, 2024. [Accessed 16-12-2024].
[23] I. Amazon Web Services, "Amazon sagemaker ground truth." https://aws.amazon.com/sagemaker/groundtruth/. Accessed: 2024-7-14.
[24] I. Amazon Web Services, "Aws identity and access management." https://aws.amazon.com/iam/. Accessed: 2024-7-14.
[25] I. Novogroder, "What is llmops? key components & differences to mlops." https://lakefs.io/blog/llmops/, Mar 2025. Accessed: 2025-03-10.
[26] D. Biswas, D. Chakraborty, and B. Mitra, "Responsible llmops: Integrating responsible ai practices into llmops," 2022.
[27] M. Kazmi, "How Is LLMOps Different From MLOps?." https://medium.com/@murtuza753/how-is-llmops-different-from-mlops-27aa309a18d6, Aug 2023. Accessed: 2025-03-10.
[28] DeepEval, *DeepEval*, 2025. Accessed: 2025-01-24.
[29] Databricks, "LLMOps: Operationalizing Large Language Models." https://www.databricks.com/glossary/llmops. Accessed: 2025-03-10.
[30] B. Memarian and T. Doleck, "Fairness, accountability, transparency, and ethics (fate) in artificial intelligence (ai) and higher education: A systematic review," 1 2023.
[31] T. Chen, "Challenges and opportunities in integrating llms into continuous integration/continuous deployment (ci/cd) pipelines," in *2024 5th International Seminar on Artificial Intelligence, Networking and Information Technology, AINIT 2024*, pp. 364–367, 2024.
[32] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," *Advances in neural information processing systems*, vol. 28, 2015.
[33] DVC, "Data version control." https://dvc.org/, 2024. [Accessed 16-12-2024].
[34] Git, "Git." https://git-scm.com/, 2024. [Accessed 16-12-2024].
[35] S. Garg, P. Pundir, G. Rathee, P. Gupta, S. Garg, and S. Ahlawat, "On continuous integration / continuous delivery for automated deployment of machine learning models using mlops," in *2021 IEEE Fourth International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*, pp. 25–28, 2021.
[36] U.S. Bureau of Labor Statistics, "Occupational employment and wage statistics." https://www.bls.gov/oes/current/oes_nat.htm, 2024. [Accessed 16-12-2024].
[37] D. Lake, "Delta lake." https://delta.io/. [Accessed 16-12-2024].
[38] Wekan, "Wekan - open-source kanban." https://wekan.github.io/, 2024. [Accessed 16-12-2024].
[39] Taiga, "Taiga: The free and open-source project management tool." https://taiga.io/, 2024. [Accessed 16-12-2024].
[40] Microsoft, "Azure machine learning." https://azure.microsoft.com/en-us/products/machine-learning, 2024. [Accessed 16-12-2024].
[41] Amazon Web Services, "Aws lake formation." https://aws.amazon.com/lake-formation/, 2024. [Accessed 16-12-2024].
[42] Asana, "Asana." https://asana.com/, 2024. [Accessed 16-12-2024].
[43] ClickUp, "Clickup." https://clickup.com/, 2024. [Accessed 16-12-2024].
[44] Atlassian, "Trello." https://trello.com/, 2024. [Accessed 16-12-2024].
[45] Red Hat, "Ansible collaborative." https://www.ansible.com/, 2024. [Accessed 16-12-2024].
[46] HashiCorp, "Terraform community." https://www.terraform.io/, 2024. [Accessed 16-12-2024].
[47] Microsoft, "What is azure resource manager?." https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/overview, 2024. [Accessed 16-12-2024].
[48] Microsoft, "Bicep documentation." https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/, 2024. [Accessed 16-12-2024].
[49] Progress, "Chef." https://www.chef.io/, 2024. [Accessed 16-12-2024].
[50] Puppet, "perforce puppet." https://www.puppet.com/, 2024. [Accessed 16-12-2024].
[51] PyTorch, "Pytorch." https://pytorch.org/, 2024. [Accessed 16-12-2024].
[52] TensorFlow, "Tensorflow." https://www.tensorflow.org/, 2024. [Accessed 16-12-2024].
[53] Pandas Team, "pandas." https://pandas.pydata.org/docs/, 2024. [Accessed 16-12-2024].
[54] JUnit Team, "Junit 5." https://junit.org/junit5/, 2024. [Accessed 16-12-2024].
[55] Pytest Team, "pytest." https://docs.pytest.org, 2024. [Accessed 16-12-2024].
[56] MosaicML, "composer." https://github.com/mosaicml/composer/, 2021. [Accessed 16-12-2024].
[57] H2O.ai, "H2o.ai." https://h2o.ai/, 2024. [Accessed 16-12-2024].
[58] IBM, "Ai fairness 360 (aif360)." https://github.com/Trusted-AI/AIF360, 2024. [Accessed 16-12-2024].
[59] Amazon Web Services, "Amazon sagemaker." https://aws.amazon.com/sagemaker/, 2024. [Accessed 16-12-2024].
[60] Amazon Web Services, "Amazon kinesis." https://aws.amazon.com/kinesis/, 2024. [Accessed 16-12-2024].
[61] Neptune.ai, "Neptune.ai." https://neptune.ai/, 2024. [Accessed 16-12-2024].
[62] Weights & Biases, "Weights & biases." https://wandb.ai/site, 2024. [Accessed 16-12-2024].
[63] LangChain, "Langchain." https://www.langchain.com/, 2024. [Accessed 16-12-2024].
[64] LlamaIndex, "Llamaindex." https://www.llamaindex.ai/, 2024. [Accessed 16-12-2024].
[65] DeepAI, "Deepai." https://deepai.org/, 2024. [Accessed 16-12-2024].
[66] LM Studio, "Lm studio." https://lmstudio.ai/, 2024. [Accessed 16-12-2024].
[67] Feast, "Feature store for production ml." https://feast.dev/, 2024. [Accessed 16-12-2024].
[68] Docker, "Docker hub." https://www.docker.com/products/docker-hub/, 2024. [Accessed 16-12-2024].
[69] NVIDIA, "Nvidia ngc." https://www.nvidia.com/en-us/gpu-cloud/, 2024. [Accessed 16-12-2024].
[70] Atlassian, "Code and ci/cd." https://bitbucket.org/, 2024. [Accessed 16-12-2024].
[71] Microsoft, "Azure container registry." https://learn.microsoft.com/en-us/azure/container-registry/, 2024. [Accessed 16-12-2024].
[72] Hugging Face, "Hugging face." https://huggingface.co/, 2024. [Accessed 16-12-2024].
[73] Microsoft, "Azure devops." https://azure.microsoft.com/en-us/products/devops, 2024. [Accessed 16-12-2024].
[74] GitLab, "Gitlab for enterprise." https://about.gitlab.com/enterprise/, 2024. [Accessed 16-12-2024].
[75] BentoML, "Openllm." https://github.com/bentoml/OpenLLM, 2024. [Accessed 16-12-2024].
[76] TensorFlow, "Tensorflow serving." https://github.com/tensorflow/serving, 2024. [Accessed 16-12-2024].
[77] Jenkins, "Jenkins." https://www.jenkins.io/, 2024. [Accessed 16-12-2024].
[78] Seldon, "Seldon." https://www.seldon.io/, 2024. [Accessed 16-12-2024].
[79] Elastic, "Elastic stack." https://www.elastic.co/elastic-stack/, 2024. [Accessed 16-12-2024].
[80] Prometheus, "Prometheus." https://prometheus.io/, 2024. [Accessed 16-12-2024].
[81] Grafana Labs, "Grafana." https://grafana.com/, 2024. [Accessed 16-12-2024].
[82] TraceLoop, "What is openllmetry?." https://www.traceloop.com/docs/openllmetry/introduction, 2024. [Accessed 16-12-2024].
[83] Grafana Labs, "Grafana cloud." https://grafana.com/products/cloud/, 2024. [Accessed 16-12-2024].
[84] WhyLabs, "Whylabs." https://whylabs.ai/, 2024. [Accessed 16-12-2024].
[85] Datadog, "Datadog." https://www.datadoghq.com/, 2024. [Accessed 16-12-2024].
[86] Langchain, "Langsmith." https://www.langchain.com/langsmith, 2024. [Accessed 16-12-2024].
[87] A. Ali, R. Pinciroli, F. Yan, and E. Smirni, "Batch: Machine learning inference serving on serverless platforms with adaptive batching," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15, 2020.
[88] V. e. a. Reddi, "Mlperf inference benchmark," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 446–459, 2020.
[89] N. Ferry, A. Rossini, F. Chauvel, B. Morin, and A. Solberg, "Towards model-driven provisioning, deployment, monitoring, and adaptation of multi-cloud systems," in *2013 IEEE Sixth International Conference on Cloud Computing*, pp. 887–894, 2013.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 15,2026 at 20:12:43 UTC from IEEE Xplore. Restrictions apply.